

Trabajo Práctico

Generación de Archivos en Python

Integrantes

- Federico Flores
- Leandro Demaria

Materia

Programación

Tema Principal

Generación de Archivos en Python

Librerías Analizadas

- openpyxl
 - python-docx
 - ReportLab
 - PyPDF2
-

Introducción

La generación automática de archivos es una de las aplicaciones más utilizadas de Python en entornos empresariales, administrativos y técnicos.

Muchas organizaciones necesitan crear documentos de manera repetitiva, como planillas de Excel, informes de Word, presupuestos en PDF o documentación comercial. Realizar estas tareas manualmente consume tiempo y aumenta la probabilidad de errores.

Python ofrece numerosas librerías que permiten automatizar estos procesos, facilitando la creación, modificación y gestión de archivos en distintos formatos.

En este trabajo se analizan cuatro librerías ampliamente utilizadas para la generación y manipulación de documentos:

- openpyxl
- python-docx
- ReportLab
- PyPDF2

Además, se presenta un proyecto práctico de presupuestación que utiliza estas herramientas para generar documentación automáticamente.

Objetivos

Objetivo General

Investigar y demostrar el uso de librerías de Python orientadas a la generación y manipulación de archivos.

Objetivos Específicos

- Comprender el funcionamiento de openpyxl.
 - Comprender el funcionamiento de python-docx.
 - Comprender el funcionamiento de ReportLab.
 - Comprender el funcionamiento de PyPDF2.
 - Aplicar las librerías en un caso práctico real.
 - Automatizar la generación de documentos.
-

Librería openpyxl

¿Qué es openpyxl?

openpyxl es una librería de Python que permite crear, leer y modificar archivos de Microsoft Excel con extensión .xlsx.

Es una de las herramientas más utilizadas para automatizar planillas de cálculo.

Principales características

- Crear archivos Excel.
- Leer archivos existentes.
- Modificar celdas.
- Crear múltiples hojas.
- Aplicar estilos.
- Generar reportes.

Ejemplo básico

```
</> Python

from openpyxl import Workbook

libro = Workbook()

hoja = libro.active

hoja["A1"] = "Producto"
hoja["B1"] = "Cantidad"

libro.save("inventario.xlsx")
```

Uso en el proyecto

En el proyecto desarrollado se utiliza openpyxl para generar:

inventario.xlsx

Este archivo almacena:

- Productos
- Cantidades

Su finalidad es servir como control interno o inventario.

Librería python-docx

¿Qué es python-docx?

python-docx es una librería que permite crear y modificar documentos Microsoft Word desde Python.

Los documentos generados utilizan el formato .docx.

Principales características

- Crear documentos Word.
- Insertar texto.
- Agregar títulos.
- Crear tablas.
- Insertar imágenes.
- Aplicar formato.

Ejemplo básico

```
</> Python

from docx import Document

documento = Document()

documento.add_heading("Informe", level=1)

documento.add_paragraph("Contenido del informe")

documento.save("informe.docx")
```

Uso en el proyecto

Se utiliza para generar:

costos.docx

Este documento contiene:

- Costos de los productos.
- Ganancias.
- Totales.

Es un documento destinado al uso interno de la empresa.

Librería ReportLab

¿Qué es ReportLab?

ReportLab es una librería especializada en la creación de documentos PDF.

Permite generar documentos profesionales directamente desde Python.

Principales características

- Crear PDFs.
- Agregar texto.
- Insertar imágenes.
- Dibujar líneas y gráficos.
- Crear reportes personalizados.

Ejemplo básico

```
</> Python

from reportlab.pdfgen import canvas

pdf = canvas.Canvas("documento.pdf")

pdf.drawString(100, 750, "Hola Mundo")

pdf.save()
```

Uso en el proyecto

Se utiliza para generar:

presupuesto.pdf

Este documento contiene:

- Productos.
- Cantidades.
- Precio final.
- Total del presupuesto.

Es el documento que se entrega al cliente.

Librería PyPDF2

¿Qué es PyPDF2?

PyPDF2 es una librería que permite manipular documentos PDF existentes.

Puede utilizarse para:

- Leer PDFs.
- Extraer páginas.
- Dividir documentos.
- Combinar múltiples PDFs.

Ejemplo básico

```
</> Python
from PyPDF2 import PdfMerger

merger = PdfMerger()

merger.append("archivo1.pdf")
merger.append("archivo2.pdf")

merger.write("resultado.pdf")

merger.close()
```

Uso en el proyecto

Se utiliza para generar:

presupuesto_final.pdf

El documento final surge de unir:

presupuesto.pdf
+
condiciones_legales.pdf

De esta forma se obtiene un único archivo listo para enviar al cliente.

Proyecto Práctico: Sistema de Presupuestación

Descripción

Para aplicar los conceptos estudiados se desarrolló un sistema de presupuestación en Python.

El programa permite ingresar productos, cantidades y costos unitarios.

A partir de estos datos se calculan automáticamente:

- Costo total.
- Ganancia.
- IVA.
- Precio final.
- Total general.

Estructuras de Datos Utilizadas

Listas

Se utiliza una lista para almacenar todos los productos.

```
productos = []
```

Diccionarios

Cada producto se almacena en un diccionario.

```
</> Python

producto = {
    "descripcion": descripcion,
    "cantidad": cantidad,
    "costo_total": costo_total,
    "ganancia": ganancia
}
```

Ventajas de la Automatización

La automatización documental ofrece múltiples beneficios:

- Reduce tiempos de trabajo.
- Evita errores de carga manual.
- Genera documentación uniforme.
- Facilita la administración de información.
- Permite reutilizar datos para distintos documentos.

Relación con Inteligencia Artificial

Como complemento al proyecto se investigó el uso de LLMs (Large Language Models) como herramienta de apoyo al desarrollo.

Los modelos de lenguaje pueden ayudar a:

- Comprender nuevas librerías.
- Generar ejemplos de código.
- Detectar errores.
- Mejorar documentación.
- Explicar conceptos técnicos.

Sin embargo, la lógica del programa y la validación de los resultados siguen siendo responsabilidad del desarrollador.

Conclusión

Las librerías openpyxl, python-docx, ReportLab y PyPDF2 permiten automatizar la creación y manipulación de documentos de manera eficiente.

A través del proyecto desarrollado fue posible demostrar cómo una única carga de información puede utilizarse para generar múltiples documentos con distintos objetivos, reduciendo trabajo manual y mejorando la organización de la información.

Python se presenta como una herramienta muy potente para la automatización documental y la integración de procesos administrativos y comerciales.